Deep Learning for Autonomous Cars

Aishanou Rait
Carnegie Mellon University
arait@cmu@cmu.edu

Lekha Mohan
Carnegie Mellon University
lekhamohan@cmu.edu

Sai P. Selvaraj
Carnegie Mellon University
spandise@cmu.edu

Abstract

The current major paradigms for vision-based autonomous driving systems are: the mediated perception approach that parses the entire scene to make a driving decision, and the behavior reflex approach that directly maps an input image to a driving action through regression. A third paradigm Direct perception approach was proposed in [2], which maps the input image to a small number of key perception indicators that are necessary to drive safely. We use the power of transfer learning to test the direct perception approach by fine-tuning a standard AlexNet architecture pre-trained using ImageNet data. The data for training and testing are taken from the TORCS game. It is observed that the fine-tuned network performs similar to the network trained from scratch as provided by [2].

1. Introduction

Driving a car autonomously in traffic is current a hot research topic for many companies. Significant development has been in this area with Google self-driving car showing good maneuverability on open roads. The current state of the art methods use high cost devices such as laser sensors and radars in addition to vision sensors. Cameras on the other hand offer solutions which are cheaper and at the same time provide better resolution [1]. However, computer vision currently is good enough to detect all important features with reliability necessary for safe driving. Today, there are two major paradigms for vision-based autonomous driving systems: mediated perception approaches that parse an entire scene to make a driving decision, and behavior reflex approaches that directly map an input image to a driving action by a regressor. A intermediate approach– direct perception approach, is proposed in [2] which maps an image to several limited meaningful affordance indicators which are critical to safely driving through the traffic and using the parameters to control the driving action and have shown state of the art result in testing with 'The Open Racing Car Simulator' (TORCS). However the network to extract the affordances were trained from scratch on the data from TORCS, which the authors think can be speed up by using pre-trained Alex net.

Training and testing on real world data is time intensive. Also it is difficult and costly to obtain ground truth. For example in KITTI dataset ground truth was generated using Velodyne laser scanner and a GPS localization system [3] [1]. To circumvent this people are using video games to train the deep network and then test performance on the real world data. Authors in [2] have used the game TORCS (The Open Racing Car Simulator) to evaluate the performance of their system. TORCS allows the flexibility to record the ground truth for the various parameters such as distances to different lane markings, distance to the preceding car and angles between the car heading and the tangent to the road. These parameters are then fed to a logic controller which outputs the driving commands.

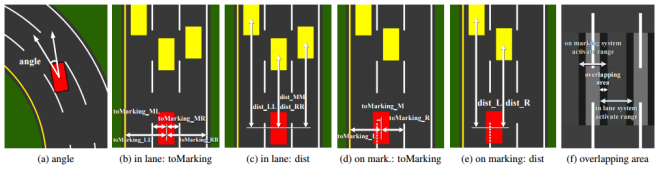
In this project we test the power of transfer learning to carry out autonomous driving in TORCS game. We use the standard AlexNet architecture pre-trained on ImageNet and fine tune it with the TORCSs generated images and ground truth labels. It is observed that the model performs comparable to the model provided by [2] in which they have trained the AlexNet architecture from scratch for 140000 iterations.

2. Related work

Two broad categories of the various approaches taken so far for autonomous driving are: the mediated approach and the behavior reflex approach.

2.1. Mediated perception

Mediated perception involves recognizing driving relevant objects such as lanes, other cars, pedestrians, traffic lights etc. The information about these individual components are then combined to form a full representation of the cars surrounding environment. Some of the information obtained through this approach is redundant because driving a car requires changing only its direction and speed. [3] In addition, obtaining accurate maps requires laser range finders, GPS, radar etc. which are costly, and getting accurate and relevant information from these sensors are still an open



(a) angle (b) in lane: toMarking (c) in lane: dist (d) on mark.: toMarking (e) on marking: dist (f) overlapping area

Figure 1. Affordance parameters

while (in autonomous driving mode)
  ConvNet outputs affordance indicators
  check availability of both the left and right lanes
  if (approaching the preceding car in the same lane)
    if (left lane exists **and** available **and** lane changing allowable)
      left lane changing decision made
    else if (right lane exists **and** available **and** lane changing allowable)
      right lane changing decision made
    else
      slow down decision made
  if (normal driving)
    $center_line$= center line of current lane
  else if (left/right lane changing)
    $center_line$= center line of objective lane
  compute steering command
  compute $desired_speed$
  compute acceleration/brake command based on $desired_speed$

Figure 2. Control logic using the affordance parameters

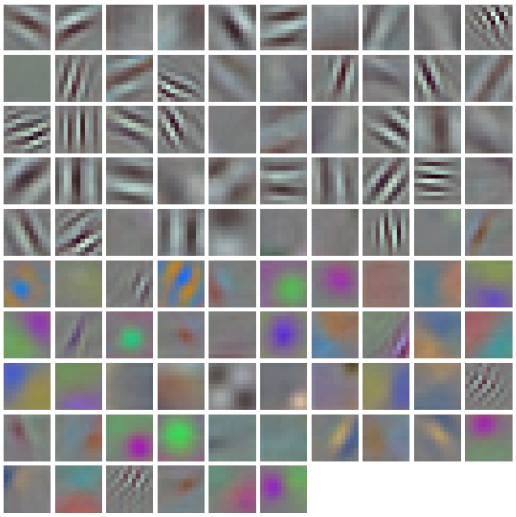


Figure 3. Conv1 filter visualization of fine-tuned network

problem.

2.2. Behavior reflex

Behavior reflex approaches use direct mapping between the image (car view) and the reaction of the driver. Though simple, this approach doesnt perform that well. This is because different drivers have different reactions to the same image which makes it very difficult for the network to differentiate between the two cases. Also this model is not able to capture the bigger picture of the situation, for example, overtaking another car, switching back to a lane are series of very low level decisions for the behavior reflex approach. This method unless trained carefully will fail to capture what is really going on [2][2].

An intermediate approach called the direct perception approach is proposed in [2] that uses affordance measures such as angle of the car relative to the road, distance to the lane markings, and distance to other cars etc. to determine driving commands for navigating a car through simulated highway scenes.

3. Direct perception approach

The Direct perception approach introduced in [2] overcomes the disadvantages by directly predicting the affordance for driving commands rather than mapping an input image to output control action or parsing indicators in an input scene. Using this approach, the input can be mapped to number of indicators like the angle of the car relative to the road, the distance to the lane markings, and the distance to cars in the current and adjacent lanes.The output of this representation serves as an input to the controller that takes driving decisions and maneuver the car.

The thirteen indicators used are described in Figure 1. Two coordinate systems were used for deriving control logic using the affordance parameter the in lane system and on marking system, which are activated under different conditions. To have a smooth transition between the parameters. The full control logic is shown in Figure 2.

4. Dataset

The training dataset consists of over 50 GB of data in the leveldb format. This dataset is available on the DeepDriving webpage. The data is generated by driving manually in the TORCS game on 7 different tracks multiple times. The original road surface in TORCS is replaced with over 30 customized asphalt textures of various lane configurations and asphalt darkness levels. There are 22 different traffic cars which are programmed to exhibit different driving behaviors to create different driving patterns. While driving the images are stored along with the ground truth labels using a script. The images are subsampled to a size of 280x210.

For testing the data is generated on the fly using TORCS which is explained in detail in the next section. [4]

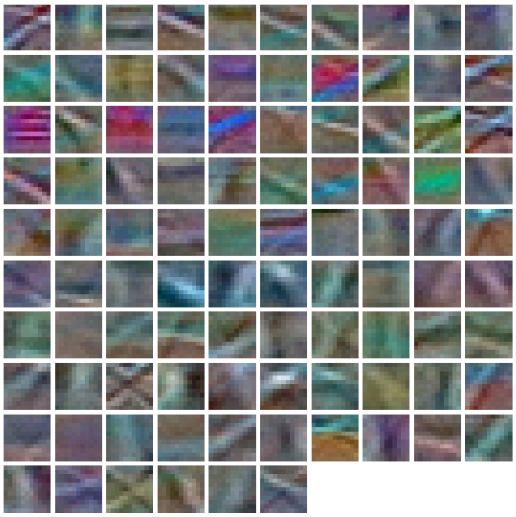


Figure 4. Conv1 filter visualization of network trained from scratch

5. Implementation

The proposed implementation of direct perception approach used Alexnet architecture followed by a fc9 layer with 14 output units (13 affordance parameters and one in-lane or on-marking indicator), trained using Euclidean loss.

We have fine tuned the standard AlexNet architecture in Caffe which was pre-trained on the ImageNet data set. Since the network described in [2] was trained using images of dimension 280X210, the authors decided to follow the same dimensional image for fine-tuning. But since the pre-trained AlexNet model is trained on ImageNet data in which the image size is 256 by 256, dimensions of weights of layers fc6, fc7, fc8 (all fully connected layers) were different and thus were trained from scratch and the remaining layers were fine tuned from the pre-trained network.

The learning rate multipliers were set as 10 and 20 for weights and bias respectively. The previous layers were fine-tuned with learning multipliers of 1 and 2 respectively. Trained using base learning of 0.005, momentum and weight decay of 0.9 at a stepsize of 8000 iterations with a weight decay of 0.0005. Training was done using a batch-size of 64, with dropouts on all the fully connected layer with a probability of 0.5.

The Caffe source code and its interface with TORCS is provided online on the DeepDriving webpage [4]. We used their code with certain modifications. They have a pre-trained available for the 3 lane configuration which is obtained by training the AlexNet model from scratch for

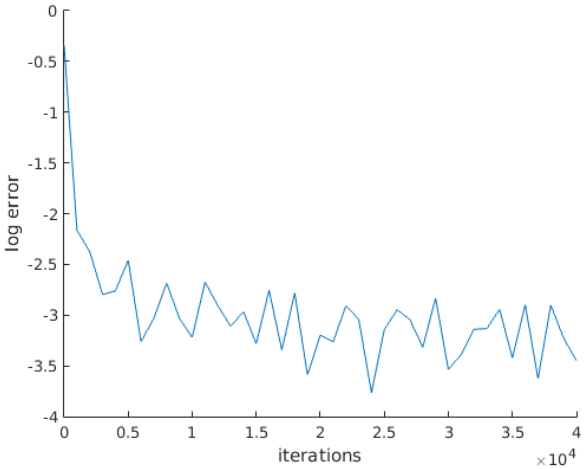


Figure 5. Fine-tuning training error of the network vs iterations

140000 iterations. We compare the performance of this network with our model trained for 40000 iterations. Convergence plot for the fine-tuning is shown in Figure 5.

For testing the data is generated in real time using the TORCS game and the output from network is used to drive the car in the simulator. There are 12 traffic which a programmed and 1 host car which is controller using the network. The TORCS view can be changed to the facing view and this is image is input to the network. The network then outputs the affordance indicators which are input to a high level controller which uses these values to compute the driving commands which include steering command, acceleration and brake. The logic of the controller is outlined in Figure 2. While driving in a particular lane the goal is to minimize the gap between the cars current position and the center-line of the lane. The center line is determined using the affordance indicators. When the car switches lanes the center-line changes from the current lane to the target lane. Screen shots of fine-tuned network driving are shown in Figure 6 and 7.

6. Results

We compared the provided model which is trained from scratch with our fine-tuned model and observed that the host car performs remarkably well while driving through traffic in 3 lane configuration. It is able to maneuver effectively near other cars and overtake them as shown in the video here. During various runs no collision was observed. One collision scenario was created deliberately by starting the host car as soon as the other cars start moving (performance of fine-tuned model on this scenario is not reported in [2] and in their demo instructions they advise to wait some time for the other cars to move before starting the host car). Since the at this time another car (blue colored) is very close to

Figure 6. Host car slowing down behind another car

Figure 7. Host car changing lane and overtaking another car

the host car the host car is not able to react in time causing a collision. After recovering from the collision it begins to proceed in the right most lane in which the red car appears. Again a collision is about to happen but instead sudden brakes are applied by the host car and the collision is avoided. This demonstrates the performance of the network. Video available here shows this case.

To quantitatively evaluate our results we modified the code to store the errors in all the affordance indicators for both the models. To ensure level ground in both cases the host car was started at almost the same time and allowed to run for a fixed amount of time. The average deviations from ground truth for both the models are shown in Figure 8. It is seen that there is not much difference in the performance which demonstrates the power of transfer learning.

Further the filter for conv 1 layer for both the fine-tuned network and the network trained from scratch [2] were visualized which are shown in Figures 3 and 4. The filters which are obtained by training from scratch It is observed that though the filters look quite different the performance is comparable. The filters from the scratch model capture straight lines and crosses which are typical of road a images. The fine-tuned model on the other hand is almost same as the ImageNet model. Some contribution to the similar performance of these models can be attributed to the edges and

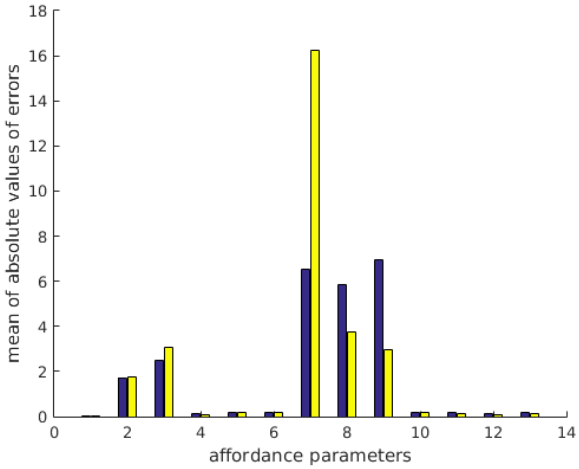


Figure 8. Mean Absolute value of errors of the predicted affordance parameter. Shown in yellow is the error by model trained by [2], and blue represents the error of the finned tuned network

boxes also learnt by the network.

7. Conclusion

From the Mean error plot Figure 8, it can be seen that both the network performed similarly in most cases and performed better in few affordances and worse in other. Hence it can be concluded that although the Imagenet was trained to represent any image, it performs similar to the network trained solely of the task at hand (predicting affordance from TORCS images). Thus we conclude that the re-usability of convolution network exceeds what was portrayed in [2] and more exploratory work has to done on this domain to understand the generalization power of convolution neural networks.

References

[1] C. S. R. U. Andreas Geiger, Philip Lenz. Welcome to the kitti vision benchmark suite!n. 2002.

[2] D. A. Pomerleau. Alvinn: An autonomous land vehicle in a neural network. Technical report, DTIC Document, 1989.

[3] B. Templeton. Cameras or lasers?, 2010.

[4] J. Xiao. Princeton vision and robotics. 2016.